Programmation avancée en C++
GIF-1003

Révision Final

Attention

Il s'agit d'extraits des notes de cours déjà fournies

- certains étudiants croiraient à tort qu'étudier seulement le présent document permet de réussir l'examen!

T. Eude

UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

Révision

- Considérations de génie logiciel
- Implantation de hiérarchies de classes
- Gestion de la mémoire
- Gestion des erreurs et des exceptions
- Frameworks
- Librairie Standard du C++ (STL)

Programmation avancée en C++
GIF-1003

Module 3 :
Considérations de génie logiciel

T. Eude

UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

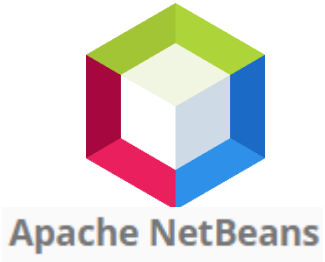
3. Considérations de génie logiciel

3.1 Environnement de développement intégré (IDE) et étude du cycle de programmation

Université Laval
Département
d'informatique

UNIVERSITÉ LAVAL

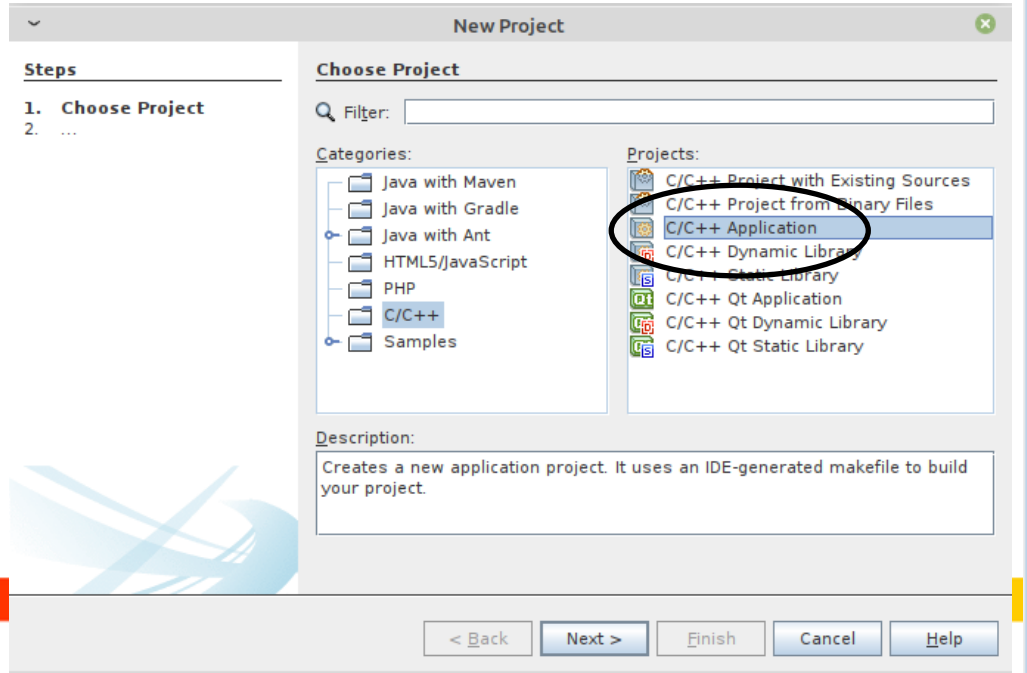
Environnement de développement intégré (IDE)

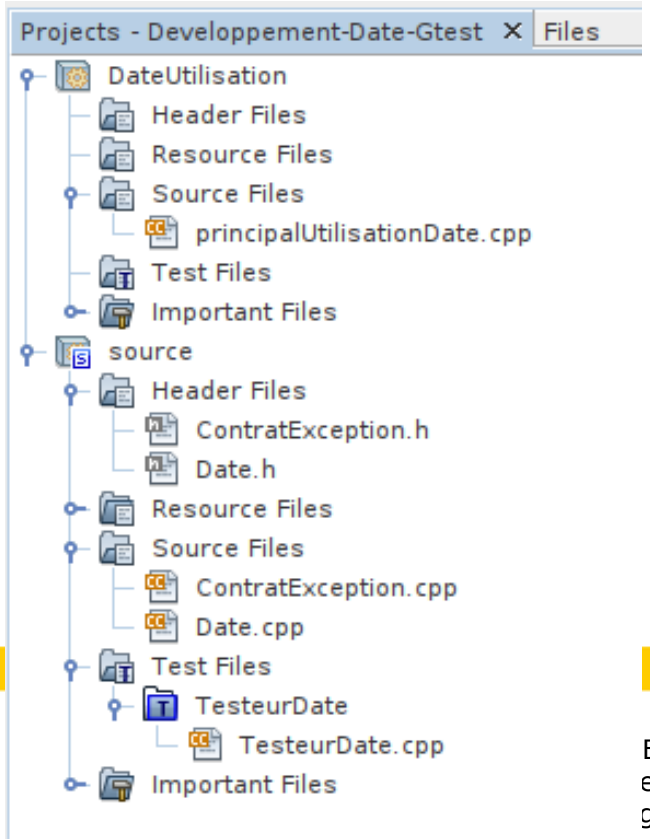


- Compilateur : g++
- Éditeur de lien : g++
- Débogueur : gdb
- Documentation : doxygen
- Interface GUI : Framework Qt
- …

Programmation par projets

- « profils » de compilation
- Développement de classes ➔ testeurs
- Intégration





3. Considérations de génie logiciel

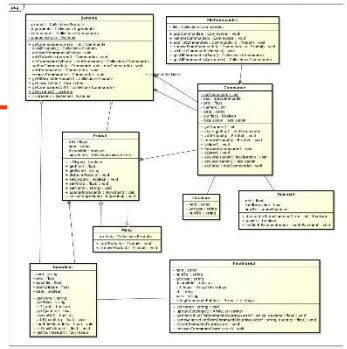
3.1 Environnement de développement intégré et étude du cycle de programmation

3.2 Modélisation : Diagrammes de classes

Université Laval
Département
d'informatique

UNIVERSITÉ LAVAL

Diagramme de classes



- Considéré comme le plus important de la modélisation orientée objet
- Montre la structure interne
- Contient principalement des classes
  - Une classe contient des attributs et des opérations
- N'indique pas comment utiliser les opérations
  - Description purement statique d'un système

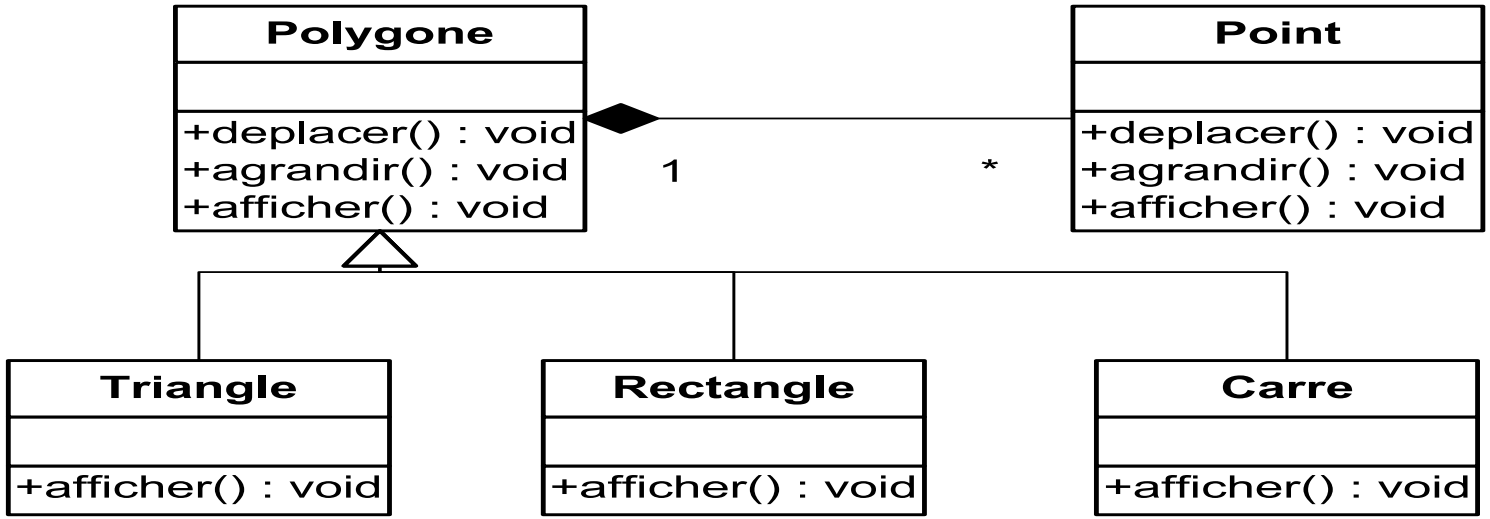
Représentation des classes

Nom de classe
NomAttribut : TypeAttribut = ValeurInitiale
NomOpération(NomArgument : TypeArgument = ValeurParDefaut, …) : TypeRetourné

- Exemple :

Voiture
-m_puissance : int = 5
+conduire(entrée conducteur)

Exemple des polygones



3.2 La théorie du contrat

Département d'informatique - Université Laval

UNIVERSITÉ LAVAL

Un contrat protège les deux parties

- Protège le client (dans ce cas ci vous-mêmes)
  - Spécifie quelles sont ses obligations.
  - En retour, est assuré d'un certain résultat.

- Protège le fournisseur (sous-contractant)
  - Spécifie le minimum requis (de la part du client):
    - le fournisseur ne peut être tenu responsable d'avoir manqué à des engagements ne figurant pas dans le contrat.

La spécification d'un contrat

- Spécification d'une interface
  - peut être vue comme un contrat entre un **client** (l'utilisateur du composant/classe) et un **fournisseur** (le composant/classe).
- Le contrat dit ce qui doit être rencontré par le client
  - pour pouvoir appeler une opération de l'interface :

    précondition.
- Le contrat dit aussi ce qui doit être réalisé par le fournisseur
  - de fait, ce qui est garanti au client
  - pour rencontrer ce qui est prévu par l'opération

    postcondition.

Pour garantir la fiabilité

- Un logiciel peut être vu comme un ensemble d'échanges de services entre les composants.
- L'approche par contrat utilise la métaphore des contrats commerciaux
  - ce qui est exigé du client qui utilise le service;
  - ce qui en contrepartie est garanti par le service.
- Les contrats permettent de spécifier de façon non ambiguë chacun des services.
  - Doit être explicite dans les spécifications.
  - Doit aussi être implanté à même le code source.

Les assertions : le contrat du logiciel

- Pour produire du logiciel correct et robuste :
  - rendre les obligations et les garanties explicites pour chaque appel de méthode.
- Mécanismes pour exprimer ces conditions :

❑ **assertions**

❑ **préconditions**

❑ **postconditions :**

- assertions s'appliquant à des méthodes en particulier,

❑ **invariants** de classe :

- assertions s'appliquant à toutes les méthodes d'une classe en tout temps.

Préconditions et postconditions

préconditions :

- conditions devant être remplies ***avant*** d'exécuter une méthode.
  - Si le contrat n'est pas rempli, la méthode n'a pas à s'exécuter. [Erreur de programmation]

postconditions :

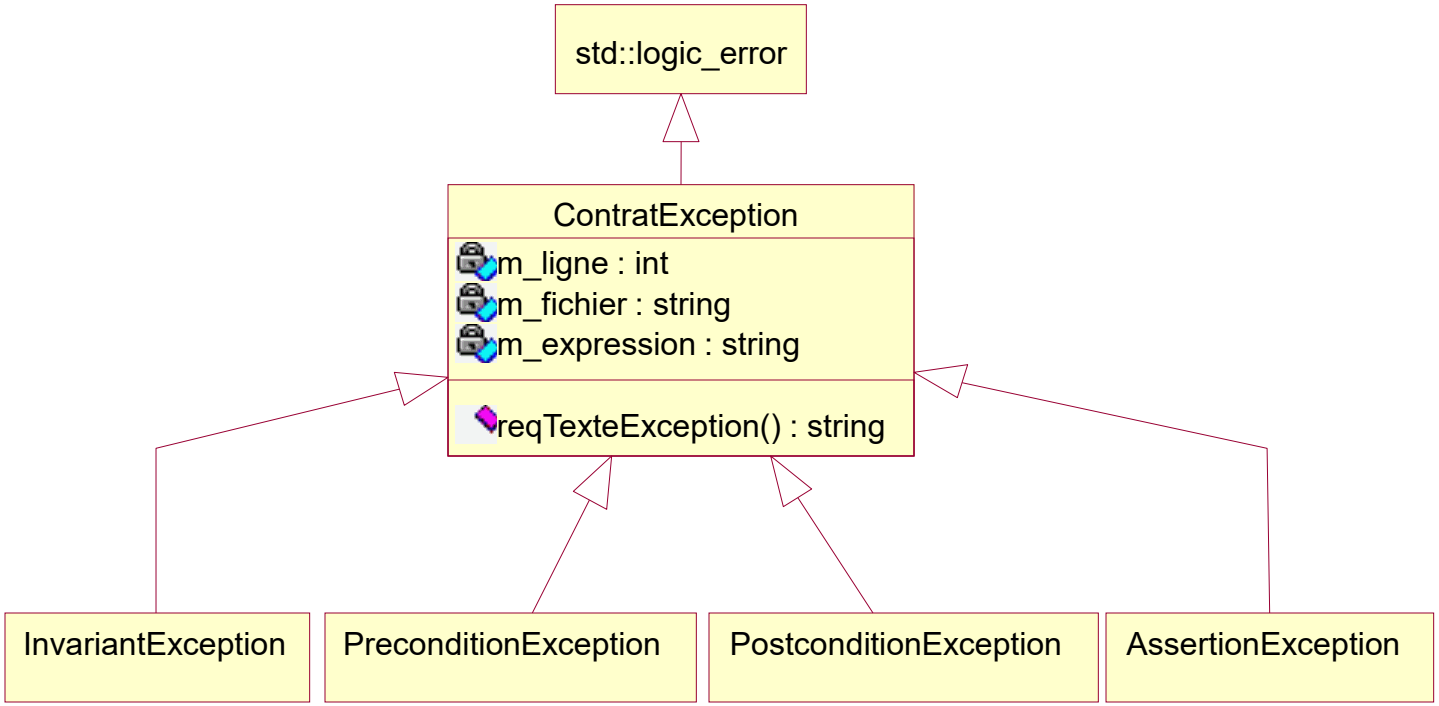
- conditions devant être remplies ***après*** l'exécution d'une méthode.
  - Si le contrat n'est pas rempli, la méthode ne doit pas retourner. [Erreur de programmation]

Invariant de classe

Invariant

- représente l'ensemble des *conditions logiques* devant être respectées pour assurer un comportement correct d'une classe.
  - Immédiatement après la création d'un objet de cette classe,
  - doit être *respecté* et *préservé* tout au long de la vie de l'objet.

Hiérarchie d'exceptions de contrat



Les MACROs pour le contrat

- "fonction" définie par un #define.
- Remplacement textuel, fait par le préprocesseur.
- Dans le fichier `ContratException.h`,
  1. Définition de ces classes
  2. + définition des MACROs suivantes:
     - `PRECONDITION(test);`
     - `POSTCONDITION(test);`
     - `ASSERTION(test);`
     - `INVARIANT(test);`
     - `INVARIANTS();`
- Regardons `ContratException.h`

Synthèse

- Règle
  - Simplicité
- Implantation hiérarchie de classes + Macro
  - PRECONDITION
  - POSTCONDITION
  - ASSERTION
  - + INVARIANT / INVARIANTS
- Pourquoi les macros?
  - désactivées en version release

3. Considérations de génie logiciel

3.1 Qualité d'une interface de classe

3.2 La théorie du contrat

3.3 Le test unitaire

Département d'informatique - Université Laval

UNIVERSITÉ LAVAL

Mise en œuvre

- Implantation d'un test unitaire sur une classe : on ne teste pas
  - n'importe quoi,
  - n'importe comment
  - sans structure.

Mise en œuvre : Principes

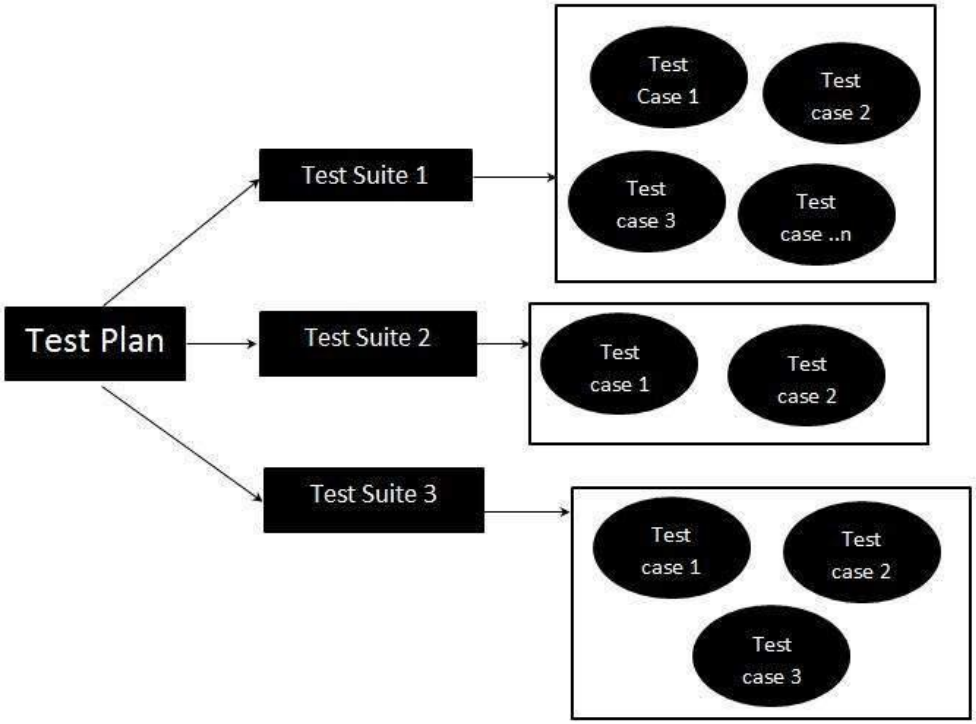
- **FIRST**
  - **F**ast (Rapide), **I**ndependent (Indépendant), **R**epeatable (Répétable), **S**elf-validating (auto-validation), **T**imely (En temps opportun)
- **AAA**
  - **A**rrange-**A**ct-**A**ssert" (Arrangement-Action-Validation)
- **Given-When-Then**
  - **Étant donné, Quand, Alors**

Google Test

- Test:
- Ensemble de fonctions (Macros) contenant des *assertions*,
  - déclarations vérifiant si une condition est vraie.
  - résultat d'une assertion
    - *success* (test réussi),
    - *nonfatal failure* (test échoué)
    - *fatal failure* (test échoué et arrête la fonction).
- Cas de tests :
  - un ou plusieurs tests,
  - regroupés selon la structure du code testé.

Google Test

- Suite de tests : pour regrouper les tests associés
- Cas de test : test unitaire (Arrangement Action Assertion)



[https://www.geeksforgeeks.org/gtest-framework/]

Programmation avancée en C++
GIF-1003

Thierry EUDE

Module 4 : Implantation de hiérarchies de classes

UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

Classe de base et classe dérivée

Dans le contexte d'héritage,

- on dit souvent qu'un objet **est une sorte** d'un autre objet.

Exemple :

un Rectangle **est une sorte** de Quadrilatère.

- Le Rectangle **hérite** d'un Quadrilatère;

✓classe Quadrilatère : **classe de base**,

✓classe Rectangle : **classe dérivée**.

Construction et destruction : Ordre d'appel

Construction des objets de la hiérarchie

- de la classe de base tout en haut jusqu'à la classe dérivée.

Destruction des objets de la hiérarchie

- de la classe dérivée jusqu'à la classe de base.

Héritage : contrôle des accès

Visibilités :

- Privée : Accès interne de la classe.
- Publique : Accès à tous les utilisateurs.
- Protégée : Accès aux héritiers.

Les méthodes d'une classe ont **toujours** accès aux attributs de la classe, peu importe la visibilité.

- les données privées ne sont manipulables que par les méthodes de la classe à laquelle les données appartiennent.

- L'héritage ne donne pas d'accès privilégié pour la visibilité privée.

Synthèse

- Héritage:
  - Déf:
  - 2 types de classes:
    - base
    - dérives
  - Implantation:
  - Construction: Base → Dévivé
  - Destruction: Dévivé → Base
  - Accès:
    - public
    - protected
    - private

Ajout de méthodes

- **Triangle** qui est une classe dérivée de **Polygone** (héritage)
  - a ***immédiatement accès*** à cette nouvelle fonctionnalité,
- **Triangle2** qui utilise **Polygone** (composition)
  - doit être ajoutée manuellement dans la classe **Triangle2**.

- Résultat intéressant d'un point de vue de gestion du code.

- tout héritier de la classe Polygone a accès à ses méthodes qu'elles soient écrites ***maintenant*** ou dans le ***futur***.

Remplacer, hériter, augmenter …

- Une classe dérivée D peut décider de prendre action sur une méthode m() de la classe de base B:
  - La classe dérivée remplace la méthode B::m() par D::m() en implantant un algorithme différent.
  - La classe dérivée hérite de B::m() sans changement.
  - La classe dérivée augmente la méthode B::m() par une méthode D::m() qui appelle d'abord B::m() avant de faire d'autre tâches.

Synthèse

- Héritage vs composition
- Remplacer, hériter, augmenter …

Différentes formes

- En C++, trois formes :
  - Polymorphisme ad hoc :
    - Surcharge de méthode(déjà vu)
  - Polymorphisme paramétrisable :
    - Les templates
  - Polymorphisme pur

Le polymorphisme pur

- Méthodes virtuelles, alliées à l'héritage:
  - permet de rendre les programmes extensibles par l'héritage de comportements existants
  - ✓ajout de comportements spécifiques au besoin.
- Traitement d'une famille de classes - une hiérarchie - de façon **générique**.
  - On n'a pas à identifier le type de la classe avec un **switch** comme dans les langages non OO.
- Traitement générique à partir de la classe de base : **méthodes virtuelles**.
  - à l'échelle d'une hiérarchie de classes.

Exemple des polygones …

- Tous les héritiers devront supporter le protocole imposé par la classe **Polygone**.
- L'implantation de ces méthodes pourra être un peu différente d'une classe à l'autre.
- Manipulation possible d'un ensemble d'objets de différents types:
  - à partir de pointeurs sur des objets **Polygone**
  - Le programme détermine à l'exécution le type de l'objet, et donc, la méthode à appeler.

- Le mécanisme des méthodes virtuelles permet ce comportement.

Signature des méthodes virtuelles

- Une fonction virtuelle doit avoir le même prototype (même signature et même type de retour) dans toute la hiérarchie.

- Une fois qu'une fonction a été déclarée virtuelle, elle le demeure dans toute la hiérarchie à partir de ce point.

- Bonne pratique :
  - déclarer **override** les fonctions virtuelles dans les classes dérivées
    - Lisibilité
    - Vérification par le compilateur

Utilisation de la hiérarchie :

Création de polygones

```
// --- Création des Polygones...
vector<Polygone*> vPoly;

Triangle  t1(Point(10,10), Point(30,20), Point(10, 30));
Triangle  t2(Point(20,20), Point(40,20), Point(20,40));
Carre     c1(Point(50,50), 10);
Carre     c2(Point(0,0), 20);
Rectangle r1(Point(30,30), Point(60,60));
Rectangle r2(Point(5,60), Point(100,90));

vPoly.push_back(&t1);
vPoly.push_back(&r2);
vPoly.push_back(&r1);
vPoly.push_back(&c2);
vPoly.push_back(&t2);
vPoly.push_back(&c1);
```

- Création de polygones de tous types : Triangle, Carre et Rectangle.
- Vecteur servant à faire du traitement générique...
- Insérés dans un vecteur de pointeurs à des objets Polygone.

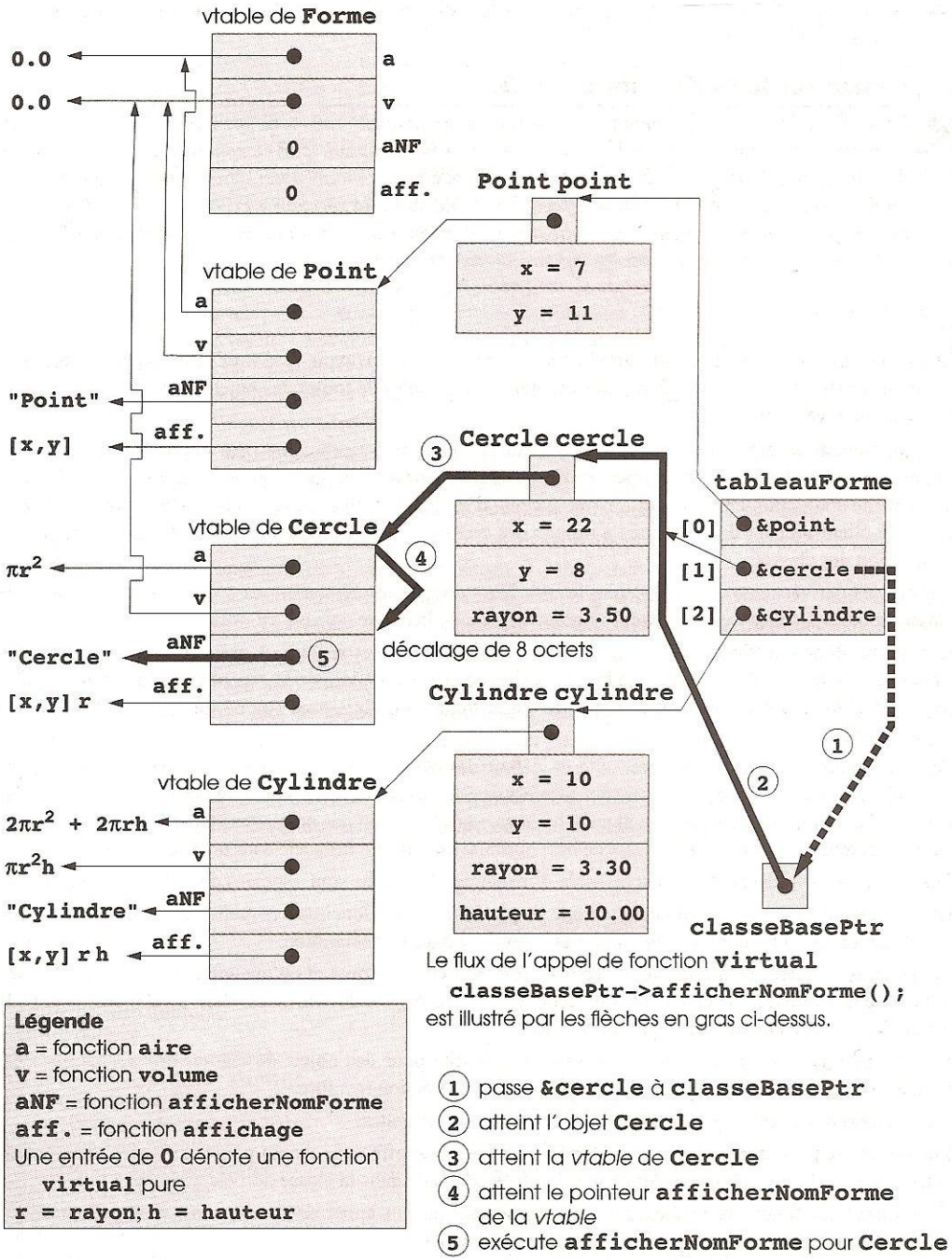
Traitement générique sur les polygones

```
// --- Utilisation polymorphique du vecteur
// --- sans connaître le type de Polygone
 for (int i=0; i<vPoly.size();i++)
   {
      vPoly[i]->afficher(cout);
      cout << endl;
   }
```

dynamiquement … à l'exécution
- détermination du type de l'objet
- appel de la bonne méthode.

Sortie :

```
Triangle  (10,10) (30,20) (10,30)
Rectangle (5,60) (100,60) (100,90) (5,90)
Rectangle (30,30) (60,30) (60,60) (30,60)
Carre     (0,0) (20,0) (20,20) (0,20)
Triangle  (20,20) (40,20) (20,40)
Carre     (50,50) (60,50) (60,60) (50,60)
```


vtable de Forme
0.0
0.0
a
v
0
aNF
0
aff.
Point point
x = 7
y = 11
vtable de Point
a
v
aNF
"Point"
aff.
[x,y]
Cercle cercle
3
tableauForme
vtable de Cercle
x = 22
y = 8
rayon = 3.50
[0] &point
[1] &cercle
[2] &cylindre
πr²
a
v
4
aNF
"Cercle"
5
aff.
[x,y] r
décalage de 8 octets
Cylindre cylindre
1
2
vtable de Cylindre
x = 10
y = 10
rayon = 3.30
hauteur = 10.00
2πr² + 2πrh
a
πr²h
v
aNF
"Cylindre"
aff.
[x,y] r h
classeBasePtr
Le flux de l'appel de fonction virtual
classeBasePtr->afficherNomForme();
est illustré par les flèches en gras ci-dessus.
Légende
a = fonction aire
v = fonction volume
aNF = fonction afficherNomForme
aff. = fonction affichage
Une entrée de 0 dénote une fonction virtual pure
r = rayon; h = hauteur
1 passe &cercle à classeBasePtr
2 atteint l'objet Cercle
3 atteint la vtable de Cercle
4 atteint le pointeur afficherNomForme de la vtable
5 exécute afficherNomForme pour Cercle

Méthodes virtuelles

- Seule une fonction membre peut être virtuelle
- Un constructeur ne peut être virtuel

- Un destructeur peut être virtuel

Classes abstraites ou concrètes

- En général :
  - Création d'une classe => créer des instances.
  - ✓Ce n'est pas toujours le cas...

- Si création d'une classe **abstraite**,
  - ➢Pas de création d'instance de cette classe.

  Exemple :

  on peut se servir d'une telle classe pour regrouper des concepts sous une classe théorique.
- Bien plus que la théorie …
  - ➢imposer une interface générale que toutes les classes héritières devront réimplanter.

Classes abstraites ou concrètes …

- En C++, pour créer une classe **purement abstraite**,
  - **au moins** une de ses méthodes déclarées purement virtuelle.

Exemple : la méthode **afficher** :

```
virtual void afficher(std::ostream&) const = 0;
```

- En mettant la méthode purement virtuelle (=0), on oblige la classe à l'être aussi.
- Le compilateur ne permettra pas de créer des instances de cette classe.

- Même dans une classe abstraite, des méthodes peuvent être implantées, comme :

```
virtual void deplacer (int p_dx, int p_dy);
virtual void agrandir (double p_facteur);
```

Programmation avancée en C++
GIF-1003

Thierry EUDE

Module 5 : La gestion de la mémoire

UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

Synthèse

- Types de mémoire
  - Zone de la pile d'appel
  - Zone d'adressage statique
  - Zone d'allocation dynamique sur le monceau
    - Celle qui nous intéresse ici (à la responsabilité du développeur)
    - pointeurs
    - new <-> delete

Fuite de mémoire

- 1 new ➔ 1 delete
- Qui a la responsabilité de libérer?

- Solution : les pointeurs intelligents

- unique_ptr
- shared_ptr
- weak_ptr

unique_ptr

- Un seul emplacement mémoire référencé par un seul pointeur
  - un unique_ptr est propriétaire de l'emplacement pointé

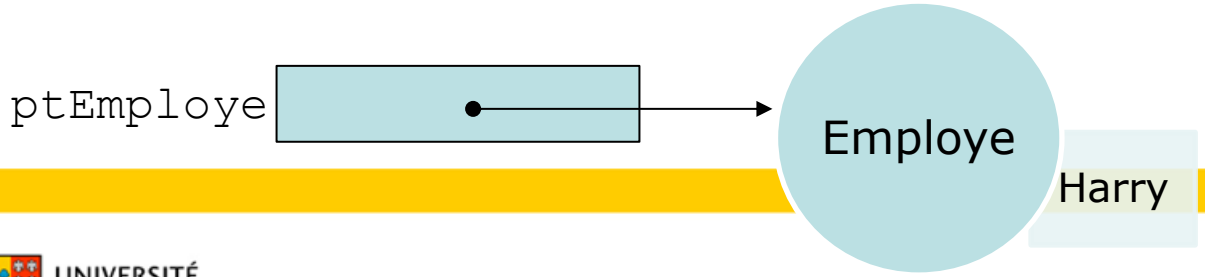
```
unique_ptr<Employe> ptEmploye (new Employe("harry"));
```

C++ 11

```
unique_ptr<Employe> ptEmploye (make_unique<Employe>("harry"));
```

C++ 14

```
auto ptEmploye (make_unique<Employe>("harry"));
```

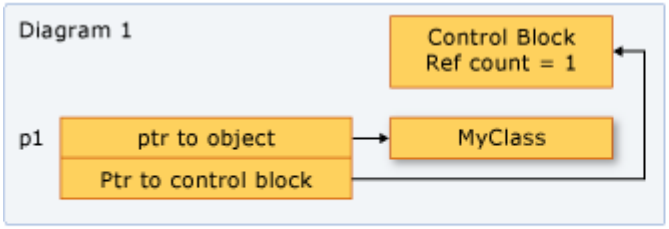


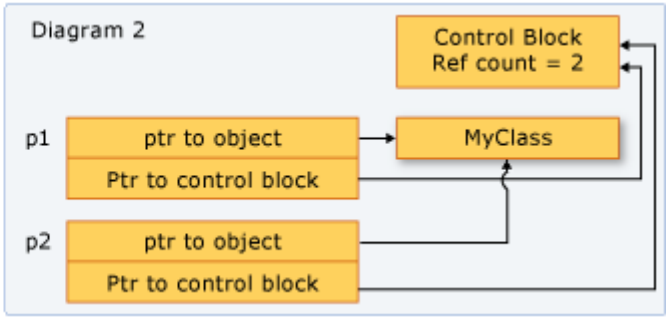
!! copie et assignement interdits

➢ erreur de compilation

shared_ptr

- Un même emplacement référencé par plusieurs pointeurs
  - Compteur de référence (Lorsque le dernier shared_ptr est détruit, l'objet pointe est également détruit.)
  - Pas de notion de propriété






```
// Utiliser la fonction make_shared lorsque cela est possible..
auto p1 = make_shared<Song>("The Beatles", "Im Happy Just to Dance With You");
// shared_ptr<Song> p1 {new Song ("The Beatles", "Im Happy Just to Dance With You")};

auto p2(sp1); // Initialisé avec le constructeur de copie. Incrémente le compteur de références.
auto p3 = p2; // Initialisé via l'affectation. Incrémente le compteur de références.
```

C++ 11

Références circulaires

Etudiant

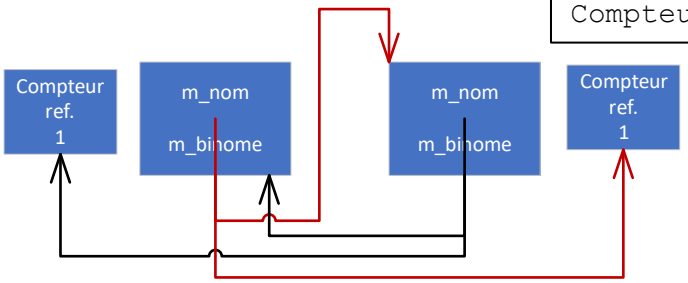
-string m_nom
~~-shared_ptr<Etudiant> m_binome~~

+ void creerBinome(std::shared_ptr<Etudiant> & p_binome)

```
auto etudiant1 = make_shared <Etudiant>("Marc");
auto etudiant2 = make_shared <Etudiant> ("Lionel");
etudiant1->creerBinome (etudiant2);
etudiant2->creerBinome (etudiant1);
```

```
Construction Etudiant Marc
Construction Etudiant Lionel
Compteur etudiant1: 1
Compteur etudiant2: 1
```

```
void creerBinome(shared_ptr<Etudiant> &
p_binome)
{
this->m_binome = p_binome;
}
```



- Solution:
  - weak_ptr

Etudiant

-string m_nom
-weak_ptr<Etudiant> m_binome

+ void creerBinome(std::shared_ptr<Etudiant> & p_binome)

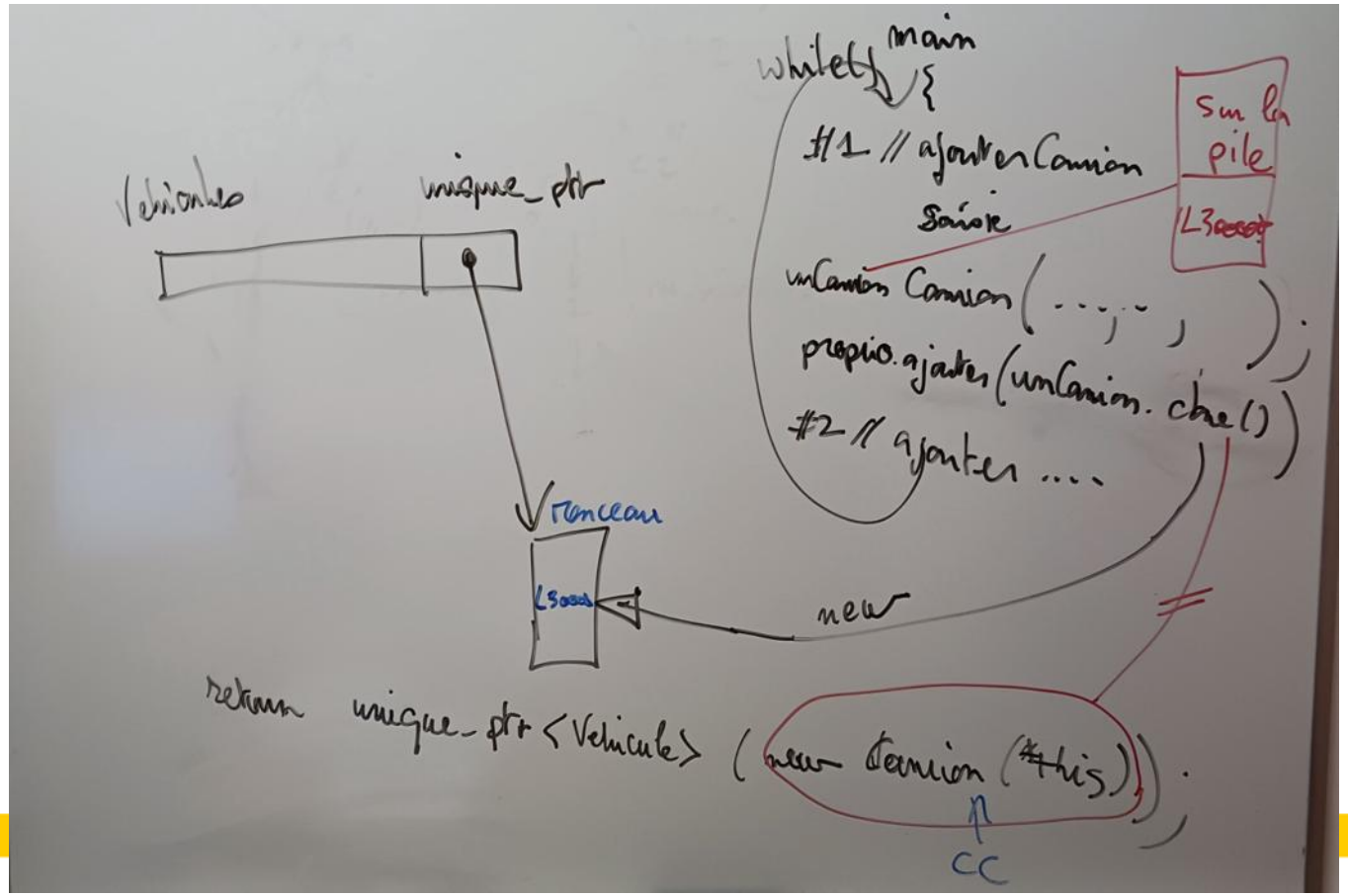
un weak pointer permet de pointer un objet géré par un shared pointer sans impacter son compteur de références.

[https://www.invivoo.com/introduction-a-la-gestion-automatique-de-la-memoire-en-c11/]

Synthèse

- Gestion de mémoire
  - déléguée aux pointeurs intelligents
    - unique_ptr (propriétaire)
    - shared_ptr (Pas de notion de propriété)
    - weak_ptr (solution aux références circulaires)

TP2 : Principe de clone



▪ Copie d'objet

- comprendre ce qui se passe lorsqu'on copie un objet sur un autre.

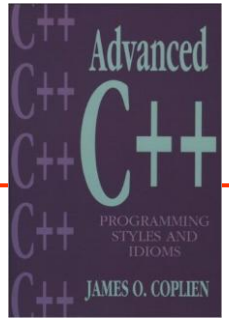
- deux sortes de copie:
  - 1. La copie de **surface** (ou shallow copy)
  - 2. La copie en **profondeur** (ou deep copy)

Si on ne fait rien de particulier,

- ➢le langage C++ fait toujours une copie de **surface** lors de la copie d'un objet.

Si on désire une copie en **profondeur**

- ➢il faut l'implanter.



En Bref : Forme canonique de Coplien

- Pour toute classe qui contient des allocations dynamique de la mémoire liées à ses attributs (notion de profondeur)
  - Utiliser des pointeurs intelligents
  - Implanter les méthodes :
    - constructeur de copie
    - surcharge de operator =

Synthèse

- Types de copie
  - de surface (par défaut)
  - en profondeur

- Gestion de mémoire supplémentaire
  - forme canonique de coplien
    - Pointeurs intelligents
    - Constructeur copie
    - Opérateur d'assignation (operator=)

Programmation avancée en C++
GIF-1003

Thierry EUDE

Module 6.Gestion des erreurs et des exceptions : Fonctionnement et utilisation

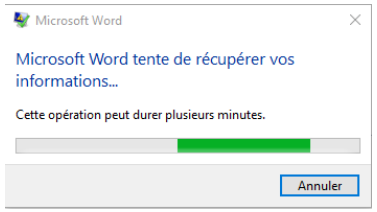
UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

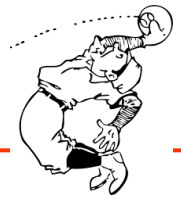
Erreurs et exceptions

- On rencontre fréquemment des erreurs ou des exceptions dans les programmes.
- Erreurs :
  - erreurs d'entrée de données par l'usager
  - erreurs du matériel ou de périphérique
  - erreurs de limitations ressources du système
  - erreurs de programmation de composants logiciels.

Réagir à ces conditions d'erreur au niveau du code

- retourner un code d'erreur
- assigner une valeur à une variable globale d'erreur
- ignorer l'erreur (!)
- imprimer un message d'erreur
- arrêter le programme
- **lancer une exception.**





• Gérer une exception

spécifiés à l'aide de blocs `try-catch`

```
try
{
    // code pouvant lancer une exception
    //avec throw Type-exception()
}
catch (Type-exception erreur)
{
    // erreur contient de l'information
sur
    // l'exception.
    // code de gestion de l'exception
}
```

Sites de détection des erreurs

interception des exceptions

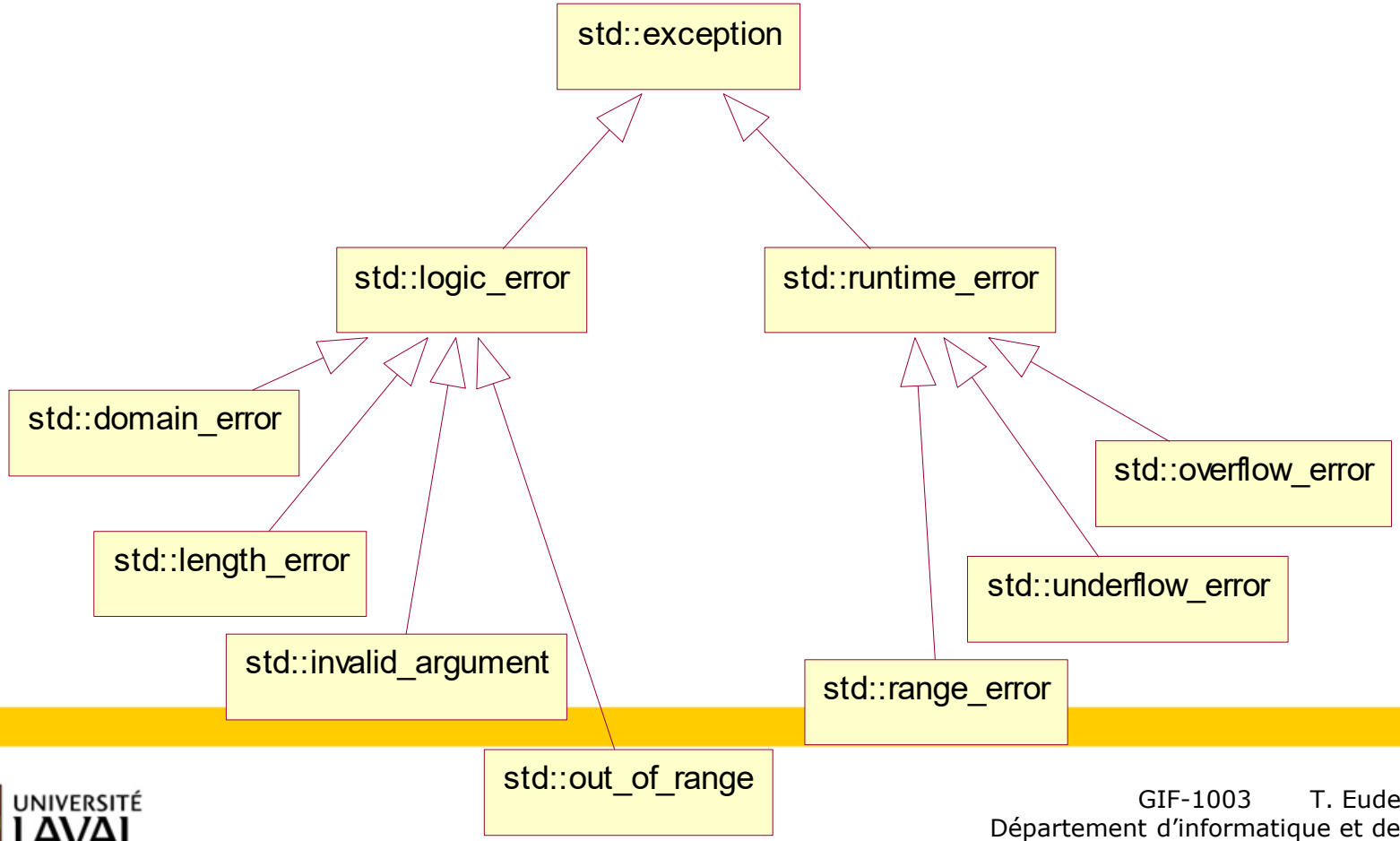
Sites de traitement des erreurs

• Hiérarchie d'exceptions

- La gestion des exceptions :
  - basée sur le type d'exception (la classe),
    - construire de petites classes appartenant à une hiérarchie.
- Librairie standard du C++ :
  - Hiérarchie d'exceptions
  - Du côté `logic_error`, i.e. erreur de programmation.
  - Du côté `runtime_error` par contre, il s'agit d'exception pouvant toujours se produire.

  - La théorie du contrat : exceptions attachées à cette hiérarchie

• Classes définies dans la librairie standard



• Gérer plus d'une exception

À partir d'un bloc `try-catch` gérer plusieurs exceptions à la fois :

```
try
{
   // Code pouvant lancer des exceptions
}
catch (overflow_error& overflow)
{
   // Gère des exceptions d'overflow
}
catch (underflow_error& underflow)
{
   // Gère des exceptions d'underflow
}
```

• Attraper toutes les exceptions et relancer

pour faire une action protectrice

pour faire un nettoyage quelconque.

```
try
{   // code
}
catch (...)
{   // Gestion de n'importe quelle exception
    throw; // Relance l'exception
}
```

pour gérer n'importe quel type d'exception

sans argument dans un bloc catch

- pour relancer l'exception courante de nouveau.

➢ toujours relancer une exception qu'on ne sait pas comment gérer (quoiqu'il aurait été préférable de ne pas l'avoir interceptée).

Une exception non interceptée arrête le programme.

Synthèse

- Exception
  - Transfert de contrôle du point de détection de l'erreur à un point de gestion, pour gérer des erreurs pouvant se produire n'importe où dans un sous-système
  - Lancer:
  - Attraper:
    - Simple:
    - Multiple:
    - Tous:
  - Ne jamais se servir des exceptions pour retourner des résultats de calcul.
  - Mécanisme puissant mais complexe (préférer des solutions plus simples si possible)
  - Utiliser des objets/pointeurs intelligents pour être « exception-safe »

Programmation avancée en C++
GIF-1003
Hiver 2011

Thierry EUDE

8. Frameworks

UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

Framework

- Impose une architecture particulière à une application.
  - la structure globale,
  - les collaborations des classes,
  - des objets
  - les tâches de contrôle d'une application générique
  - les comportements par défaut.

- Pour construire des applications rapidement.
  - sous-classage (héritage et polymorphisme) de classes abstraites.

Programmation avancée en C++
GIF-1003

Thierry EUDE

7. Classes et fonctions paramétrables, Librairie Standard du C++ (STL)

UNIVERSITÉ LAVAL
Département d'informatique
et de génie logiciel

7 : Classes et fonctions paramétrables

7.1 Les classes et les fonctions paramétrables : Les modèles (Templates)

Université Laval
Département
d'informatique

UNIVERSITÉ LAVAL

• Mécanisme d'instanciation

- Processus pour obtenir une classe utilisable à partir d'une template : l'**instanciation** de la template.

```
        instanciation          instanciation
                 type spécifique      création d'objets
template ------------> classe -------------> objet

Pile<T> ---------> Pile<string> -------> m_laPileStr

Pile<T> ---------> Pile<double> ------> m_laPileReel
```

L'interface

```
template <typename T>
class TPile
{
public:
          TPile              (int capac = 0);
   bool   estVide            () const;
   bool   estPleine          () const;
   void   init               ();
   void   push               (const T&);
   T      pop                ();
   int    reqCapacite        () const;
   T      top                () const;

private:
   void   verifieInvariant   () const;

   std::vector<T> m_laPile;
   int            m_capacite;
};
```

Ajout de cet énoncé

Le type **double** devient **T**

Les méthodes : Notes supplémentaires

- Programmation modulaire

- Définition des méthodes
  - ne sont plus dans un fichier *.cpp*
  - dans un *.hpp* inclut par le *.h*.
  - dans le *.h* après la déclaration de la classe

Compilation

- **Classes patron**

patron de classe utilisé avec de nouveaux arguments ➔nouvelle classe patron compilée.

- déclaration d'un objet de type std::vector<int> et d'un autre de type std::vector<double>

➔ Instanciation de deux classes différentes.

➔ Les objets seront de types différents,

➔ Aucun lien entre les deux (pas même une conversion).

Compilation

- **Fonctions patron**

comportement similaire avec les fonctions :

➔chaque appel d'un patron de fonction avec de nouveaux types d'arguments compile une nouvelle fonction patron.

Synthèse

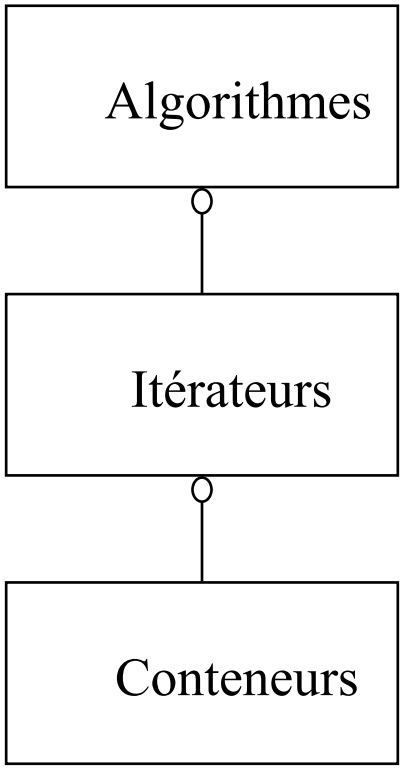
- Template
  - classe ou fonction paramétrable instanciable pour un type donné
- Instanciation
  - **`template -------> classe --------> objet`**
- Implantation
  - **`template <typename T>`**
  - **`Classe<T>::`**
  - **`T`**
- Programmation modulaire
  - .hpp inclus dans .h

7.2 Standard Template Library (STL)

Université Laval
Département
d'informatique

UNIVERSITÉ LAVAL

Structure de la librairie STL



- Composantes principales de STL:

- **Algorithmes**: définir des procédures de traitement.
- **Conteneurs**: gérer un ensemble d'espaces mémoire selon les structures de données.
- **Itérateurs**: procurer à un algorithme une façon de traverser un conteneur.

Les itérateurs

- Comme avec les pointeurs sur un vecteur C++, l'itérateur peut être incrémenté ou décrémenté pour parcourir le conteneur (`++iter ou --iter`).

- Lorsqu'on spécifie un intervalle avec deux itérateurs, `iter1` et `iter2`, il est nécessaire que `iter2` soit accessible à partir de `iter1`, i.e. un nombre fini de `++iter1` pour atteindre `iter2`.
- Les propriétés des itérateurs dépendent des conteneurs **qui les fournissent**

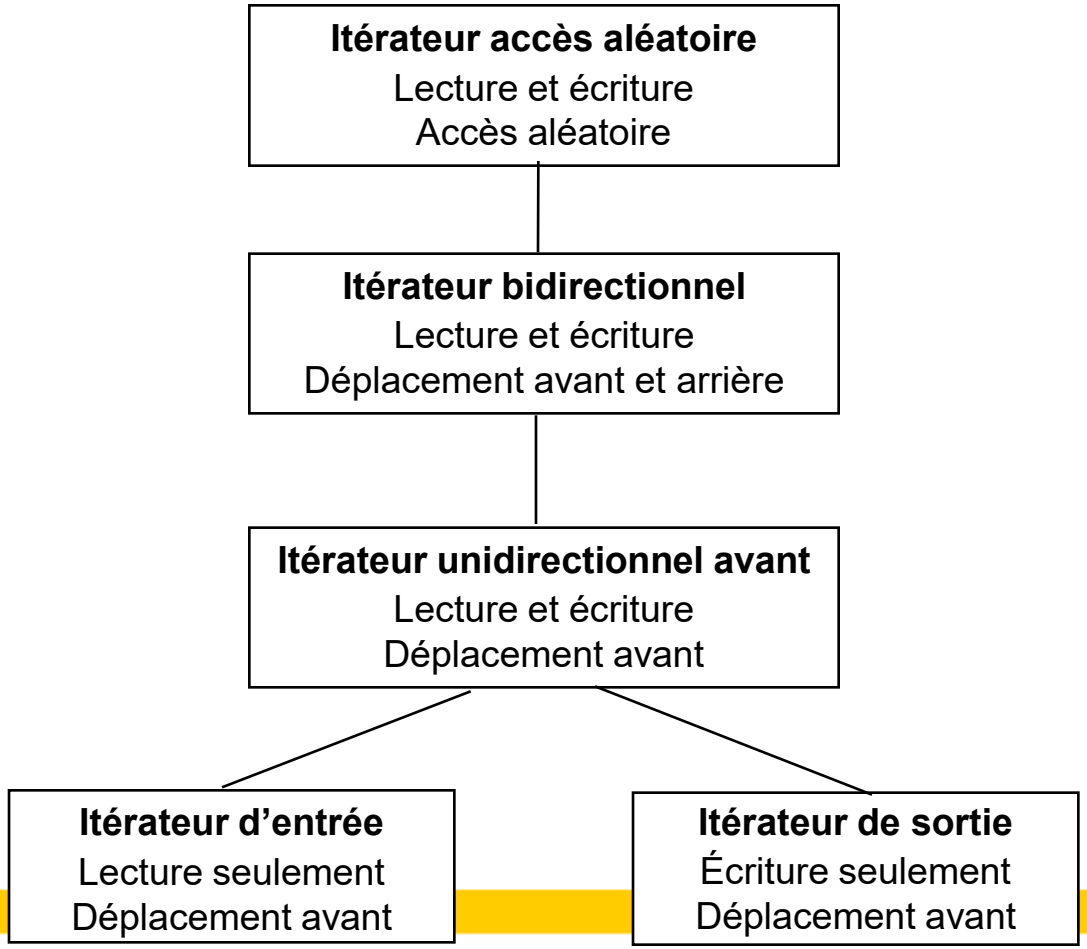
Les itérateurs

```
// ivp itérateur sur un vecteur de points
vector<point>::iterator iterateurVecteurPoints;

// itérateur sur un vecteur de pointeurs d'employés pour des
// traitements const
vector<Employe*>::const_iterator iterateurVecteurEmployes;

list<point>::iterator il ; // itérateur sur une liste de points
list<point> listePoints ;// liste de points
for (il = listePoints.begin() ; il != listePoints.end() ; ++il)
{
   // *il désigne l'élément courant de la liste de points
listePoints
}
```

Hiérarchie des itérateurs



Règle du moindre privilège

- Les itérateurs

• Les contenants dans STL

séquences	associations	adaptateurs
vector	set	stack
deque	multiset	queue
list	map	priority queue
	multimap	

Comment accéder les valeurs?

- Si l'accès aléatoire est important
  - **vector** ou **deque**
- Si l'accès séquentiel est suffisant
  - un des autres conteneurs peut être adapté.

Synthèse

- Conteneurs
  - vector
  - deque
  - list
  - set
  - multiset
  - map
  - multimap
  - stack
  - queue
  - priority_queue
  - …

- iterator
  - iterator
  - const_iterator

  - begin()
  - end()
  - cbegin()
  - cend()
  - …

• Les algorithmes génériques

- Les algorithmes fournis avec STL sont découplés du conteneur sur lequel ils agissent
  - utilisation des itérateurs.

- Tous les conteneurs dans la même catégorie peuvent utiliser les mêmes algorithmes.

Synthèse

- Algorithmes
  - découplés des conteneurs ➔itérateur
  - Types / catégories
    - Tous les conteneurs dans la même catégorie peuvent utiliser les mêmes algorithmes
    -
    -
    -
    -
    -
    -
- STL
  - + librairie majeure, performance, extensible
  - - connaître et comprendre les exigences et les limites

MERCI DE VOTRE ATTENTION

TOUS MES VOEUX DE RÉUSSITE